Algorithms Design and Analysis (CSE-243)
Asymptotic Notation

Dept. of Computer Science & Engineering

What is Asymptotic Notation?

- Asymptotic notation is a way to describe the running time or space complexity of an algorithm based on the **input size**.
- The three most commonly used notations are-
  - Big O notation (O)
  - Omega notation (Ω)
  - Theta notation (Θ)

Big O notation (O)

- Big-O notation represents the **asymptotic upper bound** of the running time of an algorithm. Therefore, it gives the worst-case complexity of an algorithm.
  - It is the most widely used notation for asymptotic analysis.
  - It specifies the upper bound of a function.
  - The maximum time required by an algorithm or the worst-case time complexity.

Big O notation (O)

- Considering **g** to be a function from the nonnegative integers into the positive real numbers.
- The O(g) is the set of function f, also from the nonnegative integers to the positive real numbers,such that

$$\mathbf{f(n) \le cg(n)} \text{ for all } \mathbf{n \ge n_o}$$

Here,

c = some real constants some : **c>0**

$\mathbf{n_o}$= some non-negative integer constant

We write $f(n) = O(g(n))$ if there exist constants $c > 0$, $n_0 > 0$ such that $0 \le f(n) \le cg(n)$ for all $n \ge n_0$.

Big O notation (O)

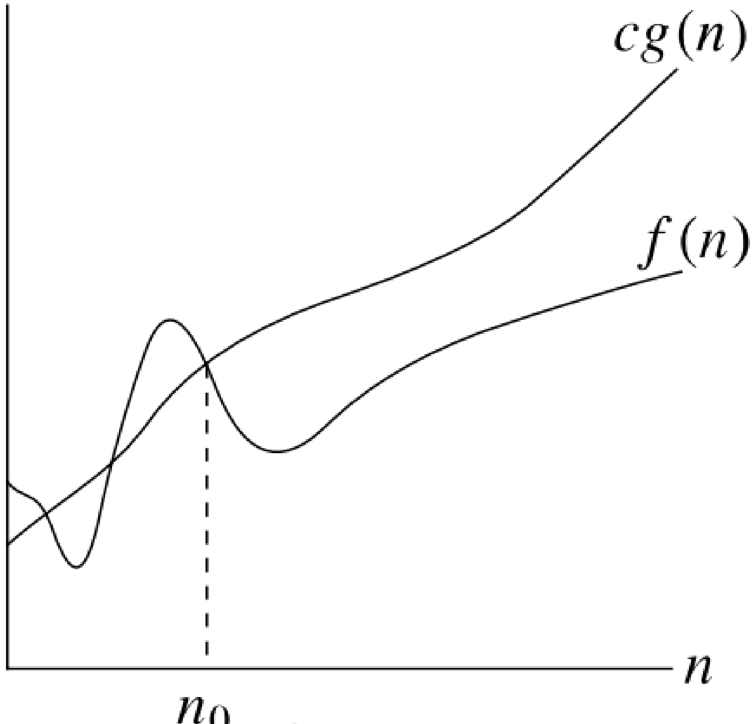
$\mathbf{f(n) \leq cg(n)}$ for all $\mathbf{n \geq n_o}$

Here,

c = some real constants some : **c>0**

$\mathbf{n_o}$= some non-negative integer constant

O(g(n)) = { f(n): there exist positive constants c and n0 such that $0 \leq f(n) \leq cg(n)$ for all $n \geq n0$ }



$f(n) = O(g(n))$

Example:

f(n) = 2n²+5 is O(n²) where c=3, n_o=5

Constant Functions

$f(n) = 16$

$f(n) = 1627$

They can be expressed as:

$f(n) \leq 16*1$ where $c=16$ and $n_o = 0$ and $g=1$

$f(n) \leq 1627*1$ where $c=1627$ and $n_o = 0$ and $g=1$

$f(n) \leq cg(n)$ for all $n \geq n_o$

$O(g(n)) = f(n)$

So $f(n) = O(1)$

Linear Function

$f(n) = 3n+5$

$f(n) = 7n+5$

For $f(n) = 3n+5$, when $n \geq 5$

$3n+5 \leq 3n+n \leq 4n$ ($c=4$, $\mathbf{n_o} = 5$)

<u>So f(n) =O(n)</u>

For $f(n) = 7n+5$, when $n \geq 5$

$7n+5 \leq 7n+n \leq 8n$ ($c=8$, $\mathbf{n_o} = 5$)

<u>So f(n) =O(n)</u>

Quadratic Function

$f(n) = 27n^2+16n$

for $n^2 \geq 16n$

$27n^2+16n \leq 27n^2+n^2 \leq 28n^2$ ($c=28$, $n_o = 16$)

So $f(n) = O(n^2)$

Cubic Function

$f(n) = 2n^3+n^2+2n$

for $n^2 \geq 2n$

$$2n^3+n^2+2n \leq 2n^3+n^2+n^2 \leq 2n^3+2n^2$$

Now

for $n^3 \geq 2n^2$

$$2n^3+2n^2 \leq 2n^3+n^3 \leq 3n^3 \qquad (c=3,\ \mathbf{n_o} = 2)$$

So $f(n) = O(n^3)$

Exponential Function

$f(n) = 2^n+6n^2+3n$

for $n^2 \geq 3n$

$$2^n+6n^2+3n \leq 2^n+6n^2+n^2 \leq 2^n+7n^2$$

Now

for $2^n \geq n^2$ $\{n \geq 4\}$

$$2^n+7n^2 \leq 2^n+7*2^n \leq 8*2^n \quad (c=8,\ \mathbf{n_o} = 4)$$

<u>**So $f(n) = O(2^n)$**</u>

Big Omega Notation (Ω)

- Omega notation represents the asymptotic lower bound of the running time of an algorithm.
- Thus, it provides the best case complexity of an algorithm

Big Omega Notation (Ω)

- Considering g to be a function from the nonnegative integers into the positive real numbers.
- The **Ω(g)** is the set of function f, also from the nonnegative integers to the positive real numbers, such that

$$\mathbf{f(n) \geq cg(n)} \text{ for all } n \geq \mathbf{n_o}$$

Here,

$\mathbf{c}$ = some real constants some : $\mathbf{c>0}$

$\mathbf{n_o}$= some non-negative integer constant

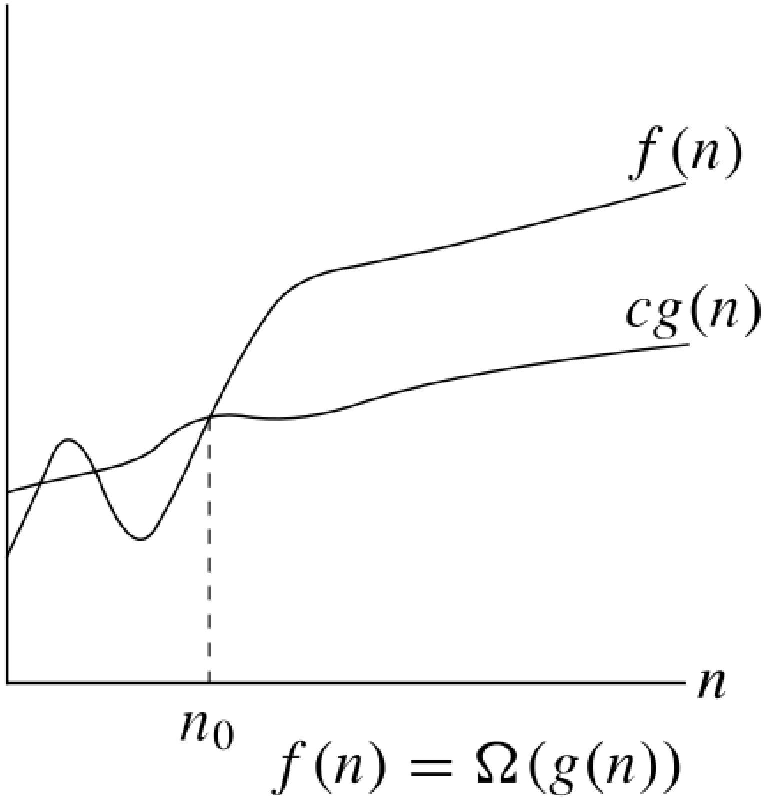
Big Omega Notation (Ω)

f(n) ≥ cg(n) for all n≥ $\mathbf{n_o}$

Here,

c = some real constants some : **c>0**

$\mathbf{n_o}$= some non-negative integer constant



Ω(g(n)) = { f(n): there exist positive constants c and n_0 such that $0 \le cg(n) \le f(n)$ for all $n \ge n_0$ }

f(n) = 2n²+5 is Ω (n²) where c=1, n_o=0

Constant Functions

$f(n) = 16$

$f(n) = 1627$

They can be expressed as:

$15*1 \le f(n) \le 16*1$ where $c_1 = 15$, $c_2 = 16$ and $n_o = 0$

$1626*1 \le f(n) \le 1627$ where $c_1 = 1626$, $c_2 = 1627$ and $n_o = 0$

$c_1 g(n) \le f(n) \le c_2 g(n)$ **for all** $n \ge n_o$

So $f(n) = \Theta(1)$ for all above function

Linear Function

$f(n) = 3n+5$

$3n < 3n+5$ for all n , $c_1 = 3$

Also

$3n+5 \le 4n$ for $n \ge 5$, $c_2 = 4$, $\mathbf{n_o} = 5$

Thus

$3n < 3n+5 \le 4n$ $\quad c_1 = 3$, $c_2 = 4$, $\mathbf{n_o} = 5$

So $f(n) = \Theta(n)$

Quadratic Function

$f(n) = 27n^2+16n$

$27n^2 < 27n^2+16n$ for all n , $c_1 = 27$

Also

$27n^2+16n \le 27n^2+n^2 \le 28n^2$ for $n \ge \mathbf{n_o} = 16$ $c_2 = 28$

Thus,

$27n^2 < 27n^2+16n \le 28n^2$ $c_1 = 27$, $c_2 = 28$, $\mathbf{n_o} = 16$

So $f(n) = \Theta(n^2)$

Cubic Function

$f(n) = 2n^3+n^2+2n$

$2n^3 < 2n^3+n^2+2n$ for all $n \geq \mathbf{n_o}, c_1 = 2$

Also,

$2n^3+n^2+2n \leq 3n^3$ for $n \geq \mathbf{n_o} = 2$ $c_2 = 3$

Thus

$2n^3 < 2n^3+n^2+2n \leq 3n^3$ $c_1 = 2, c_2 = 3, \mathbf{n_o} = 2$

So $f(n) = \Theta(n^3)$

Exponential Function

$f(n) = 2^n+6n^2+3n$

$2^n < 2^n+6n^2+3n$ for all $n \geq n_o, c_1 = 1$

Also

$2^n+6n^2+3n \leq 8*2^n$ for $n \geq n_o = 4$ $c_2 = 4$

Thus

$2^n < 2^n+6n^2+3n \leq 8*2^n$ $c_1 = 3$, $c_2 = 4$, $n_o = 4$

<u>So $f(n) = \Theta(2^n)$</u>

Big Theta Notation (Θ)

- Big Theta notation represents the upper and the lower bound of the running time of an algorithm.
- It is used for analyzing the average-case complexity of an algorithm.

Big Theta Notation (Θ)

- Considering g to be a function from the **nonnegative integers into the positive real numbers**.
- The Θ(g) is the set of function f, also from the **non-negative integers to the positive real numbers**,

  then Θ(g)= O(n) ∩ Ω(n), that means, the set of functions that are both in O(g) and Ω(n)
- The Θ(g) is the set of function 'f' such that for some positive constants c_1 and c_2 and an $\mathbf{n_o}$ exists such that,

$$c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all n, } \mathbf{n \geq n_o}$$

Big Theta Notation (Θ)

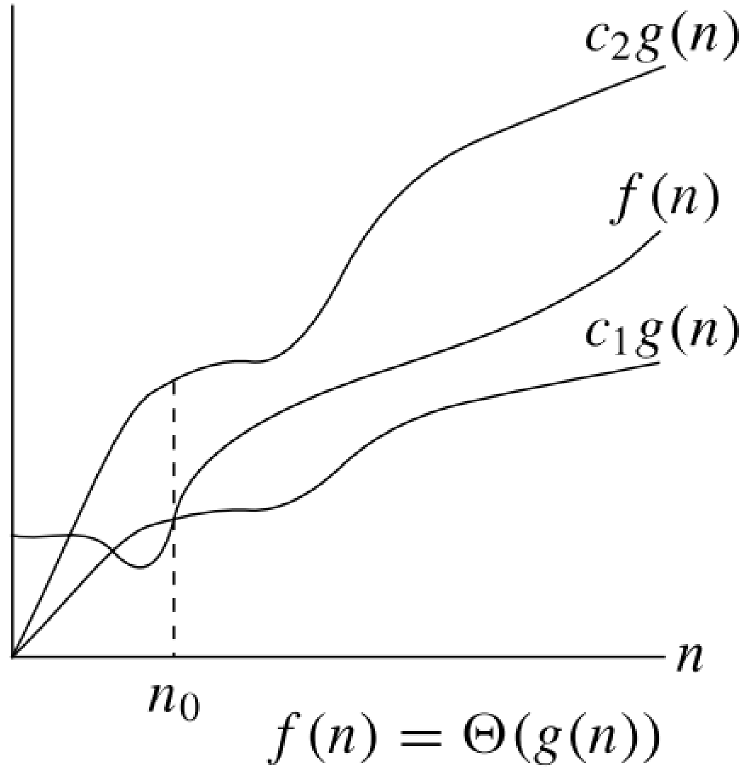
$$\mathbf{c_1 g(n) \le f(n) \le c_2 g(n)}$$

Here,

$\mathbf{c_1}$ = some positive constants some : $\mathbf{c_1 > 0}$

$\mathbf{c_2}$ = some positive constants some : $\mathbf{c_2 > 0}$

$\mathbf{n_o}$= some non-negative integer constant



Θ (g(n)) = {f(n): there exist positive constants c_1, c_2 and n_0 such that $0 \le c_1 * g(n) \le f(n) \le c_2 * g(n)$ for all $n \ge n_0$}

f(n) = 2n²+5 is Ω (n²) where c₁=1, c₂=3, n$_o$=5

Constant Functions

$f(n) = 16$

$f(n) = 1627$

They can be expressed as:

$f(n) \geq 15*1$ where $c=15$ and $\mathbf{n_o} = 0$ and $g=1$

$f(n) \geq 1626*1$ where $c=1626$ and $\mathbf{n_o} = 0$ and $g=1$

$\mathbf{f(n) \geq cg(n)}$ **for all** $\mathbf{n \geq n_o}$

$\mathbf{\Omega(g(n)) = f(n)}$

So $f(n) = \Omega(1)$

Linear Function

$f(n) = 3n+5$

$f(n) = 7n+5$

For $f(n) = 3n+5$,

$3n < 3n+5$ for all n $(c=3)$

So $f(n) = \Omega(n)$

For $f(n) = 7n+5$,

$7n < 7n+5$ $(c=7)$

So $f(n) = \Omega(n)$

Quadratic Function

$f(n) = 27n^2+16n$

$27n^2 < 27n^2+16n$ for all $n(c=27)$

So $f(n) = \Omega(n^2)$

Cubic Function

$f(n) = 2n^3+n^2+2n$

$2n^3 < 2n^3+n^2+2n$ for all $n(c=2)$

So $f(n) = \Omega(n^3)$

Exponential Function

$f(n) = 2^n + 6n^2 + 3n$

$2^n < 2^n + 6n^2 + 3n$ for all n, $(c=1)$

So $f(n) = \Omega(2^n)$

Little o and Little ω

O-notation and Ω-notation are like $\leq$ and $\geq$.
o-notation and ω -notation are like $<$ and $>$.

$o(g(n)) = \{ f(n) :$ for any constant $c > 0$,
there is a constant $n_0 > 0$
such that $0 \leq f(n) < cg(n)$
for all $n \geq n_0$ $\}$

Example: $2n^2 = o(n^3)$ $(n_0 = 2/c)$

Happy Learning